Microservices Security Architecture

Interview Guide — Vimaan (Senior Software Engineer, Software Security)

Highly detailed — 20 pages — Professional engineering diagrams

Contents

1. Executive summary
2. Architecture overview diagram and layers
3. API Gateway & Nginx (JWT validation, TLS, WAF)
4. Authentication & Tokens (JWT vs opaque)
5. mTLS and service identity
6. Service mesh and policy enforcement
7. Secrets management (Vault, KMS)
8. CI/CD and supply chain security
9. Inter-service authorization (OPA/ABAC/RBAC)
10. Data protection (SQL/NoSQL/Neo4j)
11. Logging, ELK, monitoring, SIEM
12. Threat modeling and STRIDE example
13. Rate limiting, backpressure, and DDoS mitigations
14. High availability & scaling patterns
15. Debugging scenarios & runbooks
16. Sample Nginx JWT validation snippet
17. Sample Rego policy + explanations
18. Interview question bank (mapped to JD)
19. One-page cheat sheet
20. References & next steps

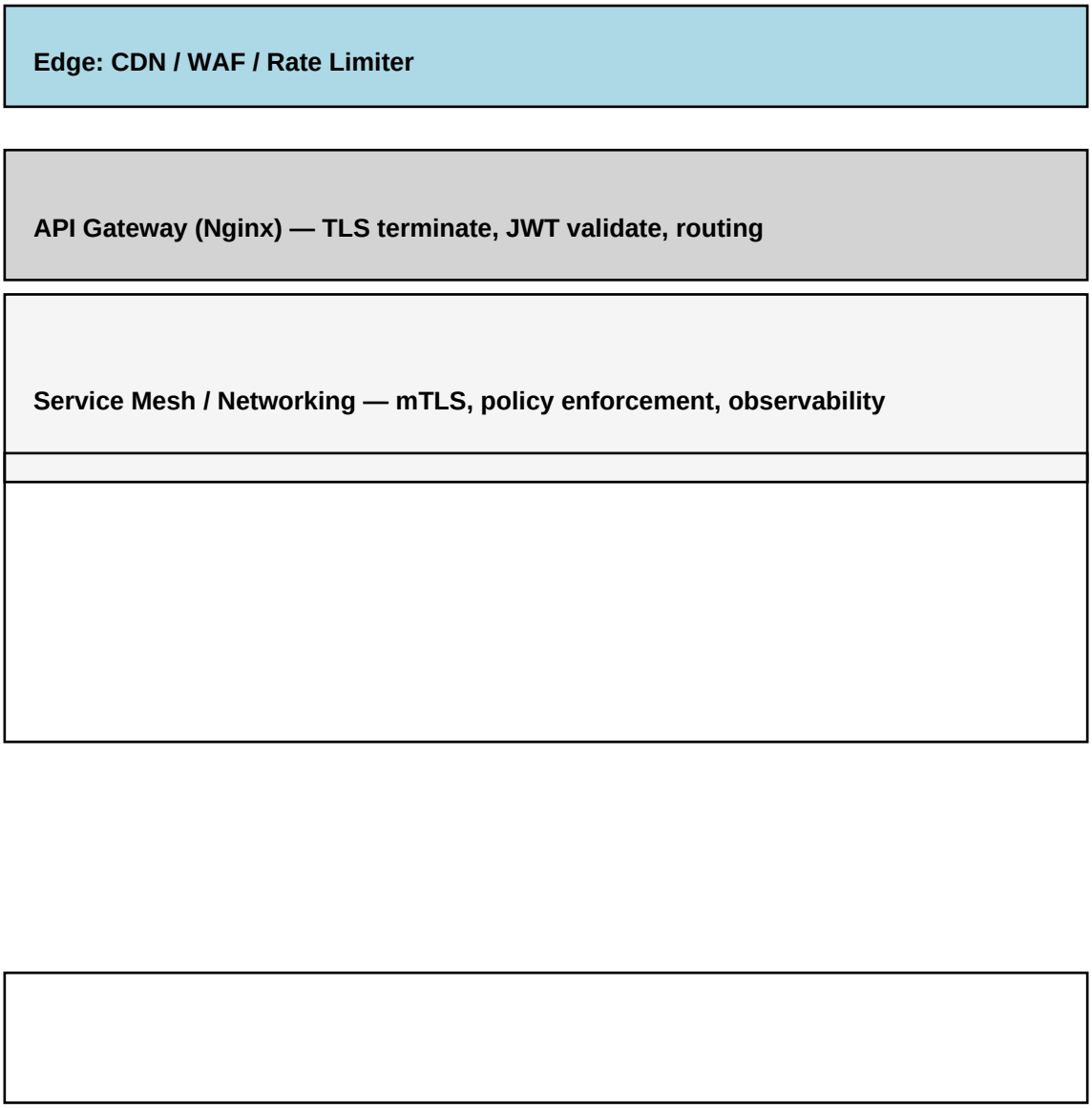
1. Executive summary

Purpose: This document prepares you to answer interview questions on secure microservices architecture, with diagrams, sample configs, threat models and debugging runbooks tailored to Vimaan's JD.

Scope: API Gateway (Nginx), JWT & OAuth2, mTLS, service mesh policies, secrets management, CI/CD security, data protection (SQL/NoSQL/Neo4j), logging/ELK, HA, and incident response.

2. Architecture overview — layers

High-level layers (outside -> inside):
- Edge: CDN / WAF / Rate limiter
- API Gateway: Nginx (authn, validation, request filtering)
- Service Mesh / Network: mTLS, policy enforcement
- Services: microservices (stateless) & stateful stores (SQL, NoSQL, Neo4j)
- Infrastructure: CI/CD, Secrets Manager, Monitoring/ELK, KMS



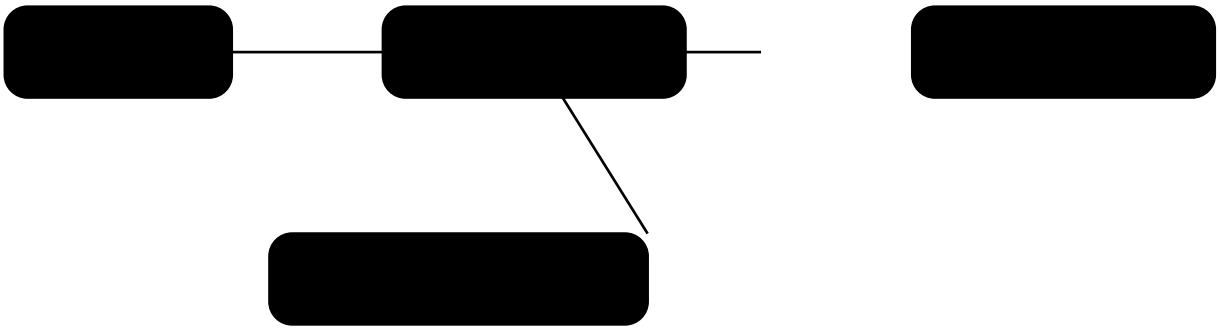
3. API Gateway & Nginx — responsibilities

API Gateway (Nginx) responsibilities:
- TLS termination and strict TLS settings (disable TLS1.0/1.1, prefer TLS1.2+)
- JWT validation (RS256 using JWKS), verify iss/aud/exp/kid
- Rate limiting, IP-based throttling, per-API quotas
- Request normalization, header whitelisting, correlation IDs
- WAF integration (ModSecurity) for OWASP protections
- Protect health endpoints and admin routes

4. Authentication & Tokens — JWT vs Opaque

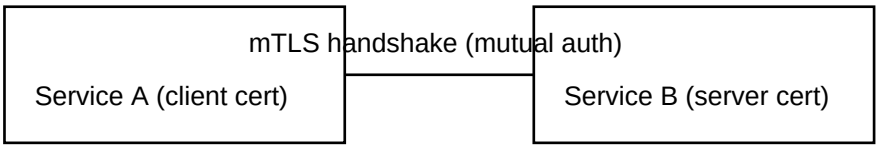
Tradeoffs:
- JWT: stateless, local verification, scalable, hard to revoke. Use short TTLs and key rotation.
- Opaque tokens: require introspection, revocable, adds latency.

Common pattern: short-lived access JWT + opaque refresh token.



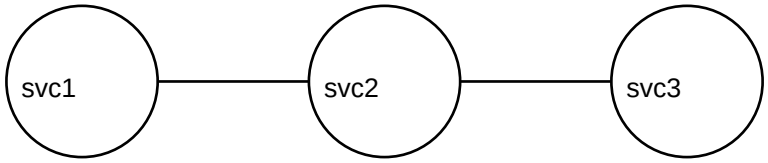
5. mTLS & service identity

mTLS ensures mutual authentication and encryption for service-to-service calls. Use short-lived certs from an internal PKI or Vault PKI. Consider SPIFFE/SPIRE for standard workload identities.



6. Service mesh (Istio/Linkerd) — policies

Use a service mesh to centralize mTLS, traffic policies, observability and enforce RBAC/ABAC at the sidecar. Mesh simplifies certificate rotation, telemetry and network policies; keep application-level auth for fine-grained checks.



7. Secrets Management — Vault / KMS

Best practices:
- Centralize secrets in Vault / cloud KMS
- Authenticate services via mTLS / IAM roles / AppRole
- Use dynamic, short-lived DB credentials
- Do not store secrets in VCS; use secretless CI/CD patterns
- Audit all secret accesses and rotate regularly

8. CI/CD and Supply Chain Security

Key controls:
- Least-privilege CI agents, protect tokens
- Secret scanning and secretless builds
- SCA (dependency scanning), static analysis
- Build artifact signing (sigstore / cosign)
- Image scanning and admission controllers to block vulnerable images
- Reproducible builds and artifact promotion gates

9. Inter-service Authorization — OPA / RBAC / ABAC

Pattern: Policy Decision Point (PDP) + Policy Enforcement Point (PEP). Use OPA (Rego) to express policies decoupled from services. Cache decisions where safe. Prefer least-privilege and attribute-based decisions for complex rules.

10. Data Protection — SQL / NoSQL / Neo4j

Protect data:
- TLS for DB connections (enable TLS on Bolt for Neo4j)
- Per-service DB users with least privilege
- Encrypt at rest with KMS-managed keys
- Parameterized queries / prepared statements to prevent injection
- Neo4j specifics: avoid exposing full graph endpoints; enable role-based access; monitor expensive queries

11. Logging & Observability — ELK / Tracing / SIEM

Centralize logs and traces. Use correlation IDs to tie requests across services. Redact PII before shipping logs. Send security events to SIEM and create detection rules for token misuse, repeated auth failures, unusual traffic patterns.

12. Threat Modeling — STRIDE

Apply STRIDE to a service: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege. Example mitigations: mTLS (spoofing), input validation (tampering), audit logs (repudiation), encryption (info disclosure), rate limits (DoS), RBAC (EoP).

13. Rate limiting, backpressure & DDoS mitigation

Implement multi-layer rate limiting: CDN -> Gateway -> Service. Use token-bucket with burst allowance. Employ bot detection, CAPTCHA for suspicious flows, and autoscaling plus circuit breakers for resilience.

14. High Availability & Scaling patterns

Design for failure: stateless services, health checks, multi-zone deployments, leader election for stateful services, read-replicas for DBs, partitioning/sharding, message queues for smoothing spikes.

15. Debugging scenario: 403 from service B when called by A

1. Capture the request and headers (Authorization, X-Trace).
2. Decode JWT: check exp, aud, iss, scopes, signature.
3. Check gateway logs for header forwarding issues.
4. Check mTLS cert validity and service identities.
5. Check service B's policy (OPA logs) for deny reason.
6. Review recent deploys/config changes and rotate creds if leakage suspected.

16. Sample Nginx (OpenResty) JWT validation — snippet

```
lua_shared_dict jwks 10m;
access_by_lua_block {
  local jwt = require "resty.jwt"
  local auth = ngx.req.get_headers()["Authorization"]
  -- extract Bearer token, fetch jwks, verify signature, check exp/iss/aud
  -- on success set ngx.req.set_header("X-User", claims.sub)
}
```

17. Sample Rego policy (OPA) — allow delete only for service-a

```
package authz

default allow = false

allow {
  input.method == "DELETE"
  input.client == "service-a"
}
```

18. Interview Q-bank — mapped to JD

- Explain end-to-end microservices security architecture you would d
- How to validate JWT in Nginx? show steps and pitfalls.
- Describe mTLS provisioning & rotation strategy.
- How to secure Neo4j for sensitive inventory graphs?
- CI/CD: how to prevent secrets in builds and ensure supply-chain int
- Design rate limits and quotas for public vs internal APIs.
- Provide an incident response plan for leaked credentials.
- Show a Rego policy to allow only authorized services to delete node

19-20. One-page cheat sheet & references

Cheat sheet:
- Edge: WAF, rate limit, TLS
- Gateway: JWT verify (iss/aud/exp/kid), whitelist headers
- Internal: mTLS + SPIFFE IDs, OPA for policies
- Secrets: Vault/KMS, dynamic creds
- CI/CD: sign artifacts (sigstore), SCA, image scanning
- DB: per-service users, TLS, KMS encryption
- Observability: correlation IDs, redact PII, SIEM alerts

References:
- OWASP Top10, NIST SP 800-63, Istio docs, HashiCorp Vault docs, OPA docs, Sigstore

Generated for interview prep. Tailored to Vimaan's JD. Good luck!